

Contents

Streaming Live RAG — 9-Day Execution Plan	1
1. Read This First — Immediate Actions	1
2. TL;DR	4
3. Non-Negotiables Cheat Sheet	4
4. Recommended Architecture	6
5. Technology Decisions (ADRs) & Final Stack	8
6. Dependency Tiers: Must-Have / Optional / Avoid	11
7. Benchmark & Test Dataset Strategy	11
8. Gate Map: G1-G6 -> Phases	12
9. Day-by-Day Phase Plan	13
10. Risks & Fallbacks	21
11. MVP vs. Advanced (MoSCoW)	22
12. Metrics Reference	23
13. Final Submission Checklist — 25 Sep	23
14. Suggested Repo Structure	24
15. Sources Consulted (17 Sep Research)	25

Streaming Live RAG — 9-Day Execution Plan

Samsung PRISM GenAI Hackathon 2026-27 · Theme 4 · 17-25 September 2026

Built strictly from Samsung_PRISM_Y2026_GenAI_Hackathon_3rd_Edition and Theme_4_Guide_RAG. Every phase, gate, and threshold below traces back to those two documents — nothing generic has been substituted in.

Team: _____ (CollegeName_TeamName) · **Theme:** 04 — Streaming Live RAG · **Submission deadline:** 25 Sep 2026, 11:59 PM IST

1. Read This First — Immediate Actions

Two things in the brief are easy to miss on a skim and will hurt you if they slip:

- **Registration closed 16 Sep 11:59 PM — that is *yesterday* relative to your 17 Sep start date.** If your team has not already submitted the registration form (<https://forms.gle/NxN6TWXLpcXmTnv66>), stop and resolve this before anything else — email prism@samsung.com immediately. The brief states the final-submission Google Form link is shared **only** with registered teams, so this single step gates everything that follows.
- **Confirm you actually have the final-submission Google Form link in hand** (organizers send it after registration closes). Don't discover on the 25th that it never arrived.

Also lock these two logistics now, not on Day 9:

- Agree on your exact **team name string** CollegeName_TeamName — it's used verbatim in your PPT filename and repo naming, and mismatches invite the “direct disqualification” clause.
- Assign the four workstreams below (compress to 2-3 people if your team is smaller than 4; nothing here requires exactly 4):

Track	Owns	Peaks on days
A — Retrieval & Data	Corpus ingestion, chunking, Qdrant, embeddings, hybrid fusion, reranking	2, 3, 8

Track	Owns	Peaks on days
B — Controller & Streaming	Chunk simulator, retrieval controller, multi-intent decomposition	4, 5, 8
C — Synthesis & Session	Grounded generation, citations, uncertainty, session memory, refinement	6, 8
D — Eval, Telemetry & DevOps	Schemas, logging, Docker, benchmark harness, demo UI, video, docs	1, 7, 9 (and continuously)

1.1 Day-by-Day Work Distribution (4-Person Team)

One person per track for the whole 9 days keeps ownership clear and context-switching low — nobody re-learns a different part of the system every morning. Pair up at the integration points called out under “Shared moments” below; everywhere else, each track runs independently against the tasks already listed in its Phase section (§9) — this is that same work, organized by person instead of by phase, so each teammate can see their own week at a glance.

Day 1 · Thu 17 (Phase 0 — Research & Architecture Lock)

- **Track A:** Run POC A — stand up Qdrant, execute one hybrid (dense+BM25) query, record latency. Start the placeholder corpus with `Doc_ID/$Section` structure.
- **Track B:** Research ADR-1 (orchestration) and the controller's decision model. Sketch the `stream_simulator.py` design ahead of Day 4.
- **Track C:** Run POC B — fire 5 sample prompts (decomposition-style and synthesis-style) at the candidate LLM provider(s); log latency and JSON validity. Draft ADR-4 and ADR-5.
- **Track D:** Scaffold the repo and `docker-compose.yml`; pin dependencies; define the 3 shared Pydantic schemas; open `RISKS.md`; confirm registration status.

Day 2 · Fri 18 (Phase 1 — Corpus & Baseline Retrieval)

- **Track A:** Finalize chunking with `Doc_ID/$Section`; build the idempotent `/ingest` script; populate Qdrant dense-only.
- **Track B:** Build the FastAPI skeleton and `/turn` request/response models, alongside Track C.
- **Track C:** Wire the `embed -> search -> prompt-stuff -> LLM` chain inside `/turn` — the deliberately naive baseline answer.
- **Track D:** Wire the `TelemetryEvent` logger into `/turn`; set up `eval/labeled_set.yaml`.

Day 3 · Sat 19 (Phase 2 — Hybrid Retrieval & Rerank)

- **Track A:** Own the day — add the native BM25 sparse vector, calibrate `avg_len`, implement hybrid prefetch+RRF, add reranking and dedup.
- **Track B:** Get ahead — draft `stream_simulator.py`'s chunk-emission logic against a few sample utterances.
- **Track C:** Get ahead — draft the synthesis prompt template and citation format against real hybrid-retrieval output.
- **Track D:** Build `bench_retrieval.py`; produce Ablation #1 draft numbers and P95 latency.

Day 4 · Sun 20 (Phase 3 — Streaming Controller -> G2)

- **Track A:** Instrument end-to-end retrieval latency so controller timing is measurable; harden any retrieval edge cases Day 3 surfaced.
- **Track B:** Own the day — finish `stream_simulator.py`; build the Wait/Retrieve/Suppress controller (heuristics + LLM classifier); log every decision.
- **Track C:** Co-write the controller's classification prompt; draft presentation-only and single-question labeled cases.

- **Track D:** Build `bench_controller.py`; run the live G2 and false-trigger measurement.

Day 5 · Mon 21 (Phase 4 — Decomposition & Parallel Retrieval -> G3)

- **Track A:** Implement cross-sub-query merge/dedup; verify retrieval is genuinely concurrent (time the `asyncio.gather` calls).
- **Track B:** Own the day — build the decomposer (structured LLM call, fragmentation guardrails, sub-query cap); wire into the controller's Retrieve path.
- **Track C:** Draft ≥ 15 compound-utterance and single-question-control labeled cases; spot-check decomposition quality by hand.
- **Track D:** Build `bench_decomposition.py`; run the live G3 and over-fragmentation measurement.

Day 6 · Tue 22 (Phase 5 — Synthesis, Grounding & Refinement -> G4, G5)

- **Track A:** Expose the per-turn retrieved-chunk-ID set cleanly — the grounding validator depends on it. Stress-test late-detail retrieval.
- **Track B:** Build the refinement classifier (`NEW_TOPIC` / `LATE_DETAIL` / `PRESENTATION_ONLY`); pair with Track C at the session boundary.
- **Track C:** Own the day — build the synthesis prompt (citations + uncertainty), the deterministic grounding validator, and `session/store.py`.
- **Track D:** Build `bench_grounding.py` and `bench_session.py`; help draft ≥ 10 late-detail and ≥ 10 presentation-only cases.

Day 7 · Wed 23 (Phase 6 — Telemetry, Reproducibility & Demo UI -> G1, G6)

- **Track A:** Make sure ingestion is part of the one-command boot, not a manual pre-step.
- **Track B:** Build the demo UI's controller-decision timeline view.
- **Track C:** Build the demo UI's streaming-answer-and-citations panel.
- **Track D:** Own the day — finalize the telemetry schema, `docker-compose.yml`, `README`, `run_eval.sh`; run the first full G1-G6 scorecard.

Day 8 · Thu 24 (Phase 7 — Hardening & Ablations)

- **Track A:** Finalize Ablation #1 (hybrid vs. dense-only) on the full eval set.
- **Track B:** Run and write up Ablation #2 (rule-based vs. model-based controller).
- **Track C:** Lead the ≥ 3 required edge-case write-ups, pulling in Track A or B whenever the edge case touches their component.
- **Track D:** Latency/cost profiling pass; re-run the full scorecard after every fix.

Day 9 · Fri 25 (Phase 8 — Final Test, Demo & Submission)

- **Track A:** Final clean-machine retest; support the retrieval/fusion segment of the demo video.
- **Track B:** Record and produce the demo video.
- **Track C:** Write the System Architecture Brief; finalize the Benchmarking & Evaluation Report.
- **Track D:** Build the PPT with the exact required naming, tag the release commit, verify the tagged commit is complete, and own the actual Google Form submission.

Shared moments (all 4, regardless of track):

- Day 1: read both source documents together; review the 5 ADRs before signing off for the day.
- Day 2: each person hand-writes ~ 4 of the 15 seed eval-set queries.
- Day 3: review Ablation #1's numbers together before calling Phase 2 done.
- Day 4: everyone contributes labeled streaming cases; check G2 together — don't start Day 5 until it's green.
- Day 5: everyone contributes compound/single-question cases; check G3 together.
- Day 6: everyone contributes refinement/suppression cases; check G4 and G5 together — this is the densest day, budget real sync time, not just a five-minute standup.

- Day 7: have whoever did *not* write the Dockerfile run the clean-machine test — that's what actually catches "works on my machine."
- Day 8: triage the scorecard together first thing; freeze scope together at day's end.
- Day 9: review everything together before the Track D lead clicks submit — nobody submits solo.

2. TL;DR

- **Build the dumbest working thing first (Day 2), then earn complexity one gate at a time.** Each of the six days from Phase 2-7 adds exactly one theme-required capability and ends with a numeric pass/fail check — you should never be more than one day away from a demoable system.
- **The single biggest architectural bet in this plan: no LangChain/LangGraph/full agent framework as the backbone.** The guide's own "Architectural Parsimony" rule explicitly grades you on justifying every component's overhead, and Day-1 research (§15) confirms multi-agent orchestration layers measurably add latency and token cost for exactly this kind of pipeline. We hand-roll the controller/decomposer/session logic in plain FastAPI + asyncio and use **Qdrant + FastEmbed** as the one piece of retrieval infrastructure worth not reinventing, because it natively does the dense+sparse hybrid fusion the theme explicitly requires.
- **The guide's own worked JSON schema (§4 of the Theme 4 guide) is not just an example — treat it as close to the required output contract.** Building your telemetry event around that exact shape from Day 1 means your observability gate (G6) and your automated benchmark harness (G1) are the *same artifact*, not two separate deliverables.
- **The hardest gate is G2 (early retrieval), not G4 (grounding).** Grounding is a deterministic check you can get to ~100% with a validator. Early retrieval requires a controller that's neither too eager (thrashing, false triggers) nor too timid (misses the 80% threshold) — it gets a full dedicated day (Phase 3) and the earliest possible numeric gate in the plan on purpose.
- **Corpus is a known unknown.** The brief says "corpus provided" but doesn't hand you one in these documents. Day 1 builds a placeholder corpus modeled on the guide's own two worked examples (workshop/venue booking, travel reimbursement policy) so nothing is blocked; swap-in is a data-only change, never a code change.

3. Non-Negotiables Cheat Sheet

Pin this next to your monitor. These come directly from Theme 4 Guide §3 and §6.

Hard engineering rules (violating these fails the theme regardless of how polished the demo looks):

Rule	What it means in practice
Corpus isolation	Every factual claim traces to the supplied corpus. No web calls, no third-party KBs, no answers from the model's parametric memory.
No hardcoding / no precomputation	The held-out replay is private. Do not embed example prompts, queries, or canned responses in code — including the three worked examples in the guide itself. Build general logic.

Rule	What it means in practice
Rigorous factual grounding	Every assertion carries a [Doc_ID §Section] tag; explicit uncertainty when the corpus doesn't cover a sub-intent.
Session-bound state	Memory lives and dies with one conversation session. No persistent profiles, no cross-session leakage.
Architectural parsimony	Every component must justify its latency/compute cost — this is your license (and the jury's expectation) to stay lean.

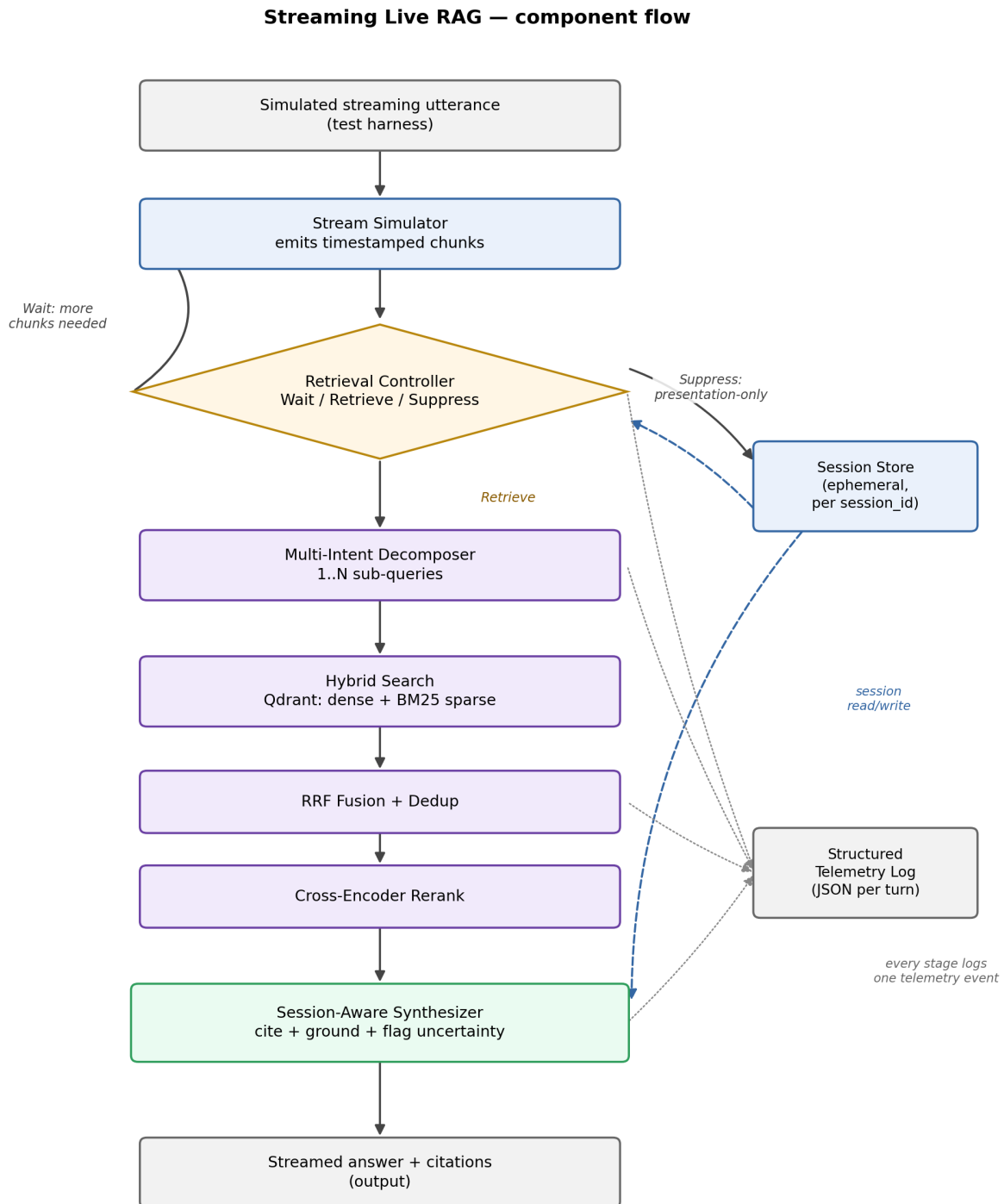
The 5 named pitfalls, and where this plan defuses each:

#	Pitfall (guide's wording)	Defused in
1	Eager/premature retrieval on noise	Phase 3 — rule-based stability heuristics + measured false-trigger rate
2	Context loss on late constraints	Phase 5 — session store + delta/versioning, never a full restart
3	Citation hallucination	Phase 5 — deterministic chunk-ID validator (not an LLM's word for it)
4	Ignoring presentation-only turns	Phase 5 — explicit <code>retrieval_required:false</code> suppression path
5	Over-fragmenting sub-queries	Phase 4 — capped sub-query count + single-question control cases in the eval set

Scoring weights, so effort goes where it pays: Working prototype & functionality 30% · Technical depth & feasibility 25% · Innovation & originality 20% · Relevance to theme 15% · Presentation & documentation 10%. Gates G1-G6 map almost entirely onto the first 55%; the two required ablations and edge-case write-ups map onto “technical depth”; executing the streaming/session-refinement mechanics well *is* the innovation the jury is grading, because it's precisely what the theme asks for — there's no separate “creativity” feature to bolt on.

4. Recommended Architecture

4.1 Component flow



4.2 Component boundaries

Component	Responsible for	Explicitly NOT responsible for	Interface
Stream Simulator	Replaying a full utterance as timestamped partial chunks (simulated ASR, as the guide explicitly permits)	Real speech-to-text	<code>iterate_chunks(utterance) -> Iterator[StreamChunk]</code>
Retrieval Controller	Wait / Retrieve / Suppress decision per incoming chunk; logs trigger + reason	Running the retrieval itself, generating answers	<code>decide(session, chunks_so_far) -> Decision</code>
Multi-Intent Decomposer	Splitting a retrieval-triggering utterance into 1-N orthogonal sub-queries	Retrieval, ranking	<code>decompose(utterance) -> list[str]</code>
Hybrid Search	Dense + sparse search and RRF fusion per sub-query (Qdrant)	Reranking, generation	<code>search(sub_query) -> list[Chunk]</code>
Reranker	Cross-encoder rerank + dedup across merged sub-query results	Retrieval, generation	<code>rerank(query, chunks) -> list[Chunk]</code>
Session-Aware Synthesizer	Grounded answer + citations + uncertainty + versioning + refinement classification (new topic / late detail / presentation-only)	Retrieval	<code>synthesize(session, evidence) -> Answer</code>
Session Store	Ephemeral per-session turns, citations, answer versions	Anything cross-session	<code>get(session_id), update(session_id, ...)</code>
Telemetry Sink	One structured JSON event per turn	Business logic	<code>emit(event: TelemetryEvent)</code>
Eval Harness	Replays the labeled set through the live API, computes the G1-G6 scorecard	Production serving	<code>run_eval(set) -> Scorecard</code>

4.3 Core data model — extend the guide's own schema, don't invent a new one

The Theme 4 guide already gives you a structured output example (§4). Treat it as the seed of your internal schema and extend it with the fields you need for telemetry and versioning:

```
{
  "session_id": "sess_8f2a",
  "turn_id": 3,
  "controller_decision": {"trigger": "multi_intent", "timestamp_s": 1.6, "reason": "..."},
  "retrieval_events": [
    {"timestamp_s": 0.8, "query": "Pune workshop venue capacity 30", "trigger": "provisional"},
    {"timestamp_s": 1.6, "query": "cancellation policy workshop venues Pune", "trigger": "multi_intent"}
  ],
  "sub_queries": ["venue capacity for 30 attendees in Pune", "cancellation terms and refund policies"],
  "answer": "...",
  "citations": ["Doc_12 §2", "Doc_31 §4"],
  "uncertainty": "Catering accommodation policies for Venue A could not be verified from the retrieved corpus.",
}
```

```

"answer_version": 1,
"latencies_ms": {"retrieval": 140, "rerank": 90, "time_to_first_token": 310, "end_to_end": 980},
"token_cost": {"input": 612, "output": 184, "usd_estimate": 0.0007}
}

```

This one object, logged once per turn, is simultaneously: your G6 telemetry record, the input your automated gate scripts read, and the raw material for your Benchmarking & Evaluation Report. Build it once, on Day 1, and never redesign it.

5. Technology Decisions (ADRs) & Final Stack

Five decisions carry real risk if made on vibes. Each is written as a condensed ADR: decision, why, alternatives seriously considered, and when you'd revisit it.

ADR-1: Orchestration — hand-rolled FastAPI/asyncio, not LangChain/LangGraph/full LlamaIndex

Decision: The controller -> decomposer -> retrieval -> synthesis -> refinement pipeline is plain Python (FastAPI + asyncio.gather for parallelism), not built on top of an agent framework.

Why: Head-to-head 2026 benchmarks of LangChain, LangGraph, LlamaIndex, and Haystack running an equivalent pipeline found LangGraph carrying the highest orchestration overhead and token usage of the group, with LangChain close behind, while Haystack and LlamaIndex ran leaner. More importantly, *none* of these frameworks has a first-class primitive for “decide whether to retrieve from an incomplete, still-arriving utterance” — that's the genuinely novel part of this theme, and you'd be writing custom code around the framework either way. Given the guide's explicit “Architectural Parsimony” rule and the fact that framework churn (deprecations, breaking API changes) is a real, currently-reported risk in this ecosystem, the lowest-risk path for 9 days is full control over the 5 custom components, wired together with nothing heavier than asyncio.

Alternatives considered: LlamaIndex as backbone (fastest path to a RAG pipeline, but its chat-engine/agent abstractions don't map to incremental-chunk-driven retrieval, so you'd fight the framework exactly where the theme is hardest). LangGraph (built for stateful multi-step agents, but that's more machinery than “controller decides retrieve-or-not, then a linear pipeline runs” needs — the guide is explicitly skeptical of this).

Revisit if: a team member has deep, current LangGraph experience and the state-machine/checkpointing primitives would demonstrably save days, not add a debugging tax.

ADR-2: Retrieval backend — Qdrant (self-hosted, one Docker service)

Decision: Qdrant, run as a single docker-compose service, is the vector database and the sparse/BM25 engine.

Why: The theme explicitly grades a “Dense/Sparse Hybrid Scoring” pipeline stage, not just similarity search. Qdrant's Query API supports named dense + sparse vectors on the same collection with **server-side BM25** and single-call fusion (RRF or DBSF) via prefetch — meaning the “hard” part of Component 3 (Corpus Retrieval & Fusion) is a well-tested, already-built primitive rather than code you write and debug under time pressure. This collapses what would otherwise be two systems (a vector index plus a separate BM25 library) into one Docker container, which directly serves both G1 reproducibility and the parsimony rule. General 2026 vector-DB guidance for hackathons often

points to FAISS as the default “just get something working” choice — but that advice assumes dense-only retrieval is enough; here, hybrid fusion quality is itself graded, so the calculus is different.

Alternatives considered: FAISS + a standalone BM25 library (bm25s, actively maintained and dramatically faster than the older rank_bm25) — genuinely fine and the recommended **fallback** if Qdrant/Docker networking becomes a blocker; you lose the one-call fusion but gain zero server dependencies. pgvector — solid in general, but adds Postgres operational overhead here for no offsetting benefit, since you don't need relational joins.

Known sharp edges (from Qdrant's own guidance): BM25's avg_len parameter is *not* computed automatically — calibrate it to your actual chunk length, or short chunks get systematically over/under-scored. Sparse learned alternatives to BM25 (BM42, SPLADE) exist but are English-only, need domain fine-tuning, and in BM42's case are explicitly unmaintained — don't reach for them under time pressure.

Revisit if: corpus turns out to be enormous (tens of millions of chunks) or multi-tenant isolation becomes a real requirement — neither is expected here.

ADR-3: Embeddings + reranker — FastEmbed (ONNX), not a PyTorch/sentence-transformers stack

Decision: qdrant-client[fastembed] for dense embeddings (BAAI/bge-small-en-v1.5, FastEmbed's own default), native Qdrant BM25 for sparse, and FastEmbed's TextCrossEncoder (ms-marco-MiniLM-L-6-v2, ONNX) for reranking.

Why: FastEmbed is built and maintained by the Qdrant team specifically to pair with Qdrant, runs on the ONNX runtime (no multi-GB PyTorch/CUDA install), and covers dense, sparse, *and* cross-encoder reranking in one lightweight package — which matters enormously for a “launches with one command on a clean machine” reproducibility gate. bge-small-en-v1.5 is a well-established, CPU-fast, MTEB-competitive 384-dim model; ms-marco-MiniLM-L-6-v2 is the widely-recommended default fast reranker (roughly 100-300ms for 50 candidates on CPU) with the best speed/quality balance for a latency-sensitive theme.

Caution, and your first honest ablation opportunity: rerankers are not automatically a win — at least one 2026 study found off-the-shelf cross-encoders *hurting* nDCG on out-of-domain corpora while adding real latency. Don't assume reranking helps; measure it on your own eval set (Phase 2) before committing, exactly as you're already required to do for the hybrid-vs-dense ablation.

Alternatives considered: BAAI/bge-reranker-v2-m3 (stronger, multilingual, heavier — a good Day-8 upgrade *if* your eval set shows it beats MiniLM and you have GPU headroom) — keep it in your back pocket, not your Day-1 default.

Revisit if: the corpus turns out to be multilingual, or a GPU is confirmed available and Day-2/3 benchmarking shows a real quality gap worth the latency.

ADR-4: LLM provider — pluggable via env var, Groq as the default for speed-sensitive calls

Decision: All LLM calls (controller classification, decomposition, synthesis) go through one thin client interface selected by an LLM_PROVIDER env var. Default recommendation: Groq's free tier (an LPU-based host serving open models at very high tokens/second and near-zero cost) for the latency-sensitive controller and decomposition calls; the team's strongest available API (Claude, GPT, or a larger Groq-hosted model) for final synthesis, where answer quality matters most.

Why: Two of the theme's own graded reporting dimensions are **time-to-first-token** and **cost per turn** — provider choice directly moves both numbers. Groq's free tier (roughly 30K tokens/minute,

~14,400 requests/day at time of writing) is generous enough for a 9-day prototype and reports first-token latency in the low hundreds of milliseconds, an order of magnitude faster than typical GPU-hosted APIs at comparable model sizes. Making this pluggable (not hardcoded) is also a direct reproducibility hedge: if a judge's clean machine doesn't have your exact API key, the README should make getting a free key a two-minute step, not a blocker.

Caution: hosted-model catalogs rotate — confirm the exact current model ID on Groq's docs on Day 1 rather than trusting any name written down in advance (e.g., `llama-3.1-8b-instant` was already retired by early 2026; check `console.groq.com/docs/models` before you build against a specific string).

Revisit if: rate limits are hit during heavy benchmark replay — Cerebras is a solid second free-tier option with a higher daily token ceiling.

ADR-5: Grounding mechanism — deterministic ID-membership check as the primary gate

Decision: G4 is measured by a fast, deterministic post-processor: parse every `[Doc_ID §Section]` tag out of the generated answer, and fail/flag any citation whose ID was not actually in that turn's retrieved-chunk set. LLM-judge-style faithfulness scoring (Ragas, DeepEval) is optional and secondary.

Why: "Zero fabricated or hallucinated document IDs" (the guide's own G4 wording) is a *verifiable* claim — you don't need an LLM's opinion to check whether a cited ID exists in a Python set. This is instant, free, and 100% reproducible, which matters because G4 is a hard pass/fail gate, not a soft quality signal. Ragas/DeepEval remain genuinely useful as a **secondary, reportable** faithfulness metric for the Benchmarking & Evaluation Report (both are actively maintained in 2026, with DeepEval favoring pytest-style CI integration and Ragas being the more RAG-specific, research-native tool) — use one of them to add rigor to the report, not to gate the pipeline.

Revisit if: you have spare time on Day 8 and want a second, LLM-graded number for the report's credibility — add it, but never let it replace the deterministic check.

5.1 Final stack at a glance

Layer	Choice	One-line reason
API/runtime	Python 3.11+, FastAPI, asyncio, uvicorn	Async-native, minimal, well understood by everyone on the team
Orchestration	Hand-rolled pipeline (no agent framework)	ADR-1
Vector + sparse DB	Qdrant (Docker)	ADR-2
Embeddings / rerank	FastEmbed (bge-small-en-v1.5 + ms-marco-MiniLM-L-6-v2)	ADR-3
LLM provider	Pluggable; Groq default	ADR-4
Grounding check	Deterministic ID-membership validator	ADR-5
Schema/validation	Hydantic	Free runtime validation for the shared TelemetryEvent/StreamChunk/SessionState contracts
Testing	pytest	Same tool for unit tests and gate-measurement scripts

Layer	Choice	One-line reason
Containerization	Docker + docker-compose	Single-command boot (G1)
Telemetry	Custom structured JSON logging	Zero external dependency, exact schema control (G6)

6. Dependency Tiers: Must-Have / Optional / Avoid

Must-have (install Day 1, no substitutes without a new ADR): fastapi, uvicorn, pydantic, qdrant-client[fastembed], fastembed, an LLM SDK matching your chosen provider (e.g. openai-compatible client for Groq, or anthropic), pytest, python-dotenv, docker + docker-compose.

Optional / experimental (add only after the MVP gates in §11 are green):

- ragas or deepeval — secondary faithfulness metric for the evaluation report.
- Langfuse (cloud or self-hosted) — nicer trace visualization for your *own* debugging and demo polish. Do **not** make it a runtime dependency of the graded system: it needs either an external account or an extra Docker service, and G1 wants a self-contained clean-machine launch.
- bm25s — only if you execute the FAISS fallback path (ADR-2) instead of Qdrant-native BM25.
- LlamaIndex's SentenceSplitter (or equivalent) — fine to import as a single chunking *utility*, never as the orchestration backbone.
- BAAI/bge-reranker-v2-m3 — quality-upgrade path for the reranker if your own ablation shows it's worth the extra latency.
- Cerebras API — backup/overflow LLM provider if Groq's free-tier limits are hit mid-benchmark.
- A minimal Streamlit or plain HTML/WebSocket demo UI — strongly recommended for the demo video, scheduled explicitly in Phase 6.

Avoid, given this theme's constraints:

- LangGraph, CrewAI, AutoGen, or any multi-agent orchestration framework as the pipeline backbone — conflicts with "Architectural Parsimony," and Day-1 research shows measurable overhead for equivalent pipelines.
- Elasticsearch/OpenSearch — heavyweight and redundant once Qdrant's native BM25 is in play.
- Any fine-tuning — no time, no need; every requirement here is achievable with prompting, retrieval, and deterministic checks.
- Building real ASR/speech-to-text — explicitly out of scope; the guide permits simulating voice input from transcripts.

7. Benchmark & Test Dataset Strategy

Corpus. You don't have the official corpus in the materials provided, so decouple it from day-to-day progress:

1. Day 1: build a small placeholder corpus mirroring the guide's own two worked domains — event/workshop venue booking (capacity, cancellation, catering) and travel/expense reimbursement policy (international travel, late-booking exceptions). These are the exact domains the guide's own examples use, so they're a reasonable stand-in for structure and citation testing even before the real corpus lands.
2. Ingestion is corpus-agnostic from the start — point it at a folder, not a hardcoded file list — so swapping in the official corpus later is a data change, never a code change.

3. Chunking assigns a stable Doc_ID (Doc_01, Doc_02, ...) and §Section marker per chunk, derived from heading structure where present, or from paragraph/fixed-size splits with synthetic section numbers otherwise. Store both the raw chunk text and its Doc_ID/§Section in the Qdrant payload — this is what makes citations checkable at all.
4. The moment the official corpus is released, re-run ingestion and the *entire* labeled eval set the same day, and diff the scorecard. Build buffer for this into Phase 7-8.

Streaming test harness. The chunk simulator (built in Phase 3) takes any full utterance string and re-emits it as a sequence of (t_offset_s, partial_text) events on a configurable cadence — this is what turns a static text eval set into “streaming” test cases without needing real voice input, which the guide explicitly permits.

Labeled eval set — build incrementally, not all at once. Categorize every test case so each gate has ground truth to score against:

Category	Targets	Seed size (Day 2)	Target size (Day 7)
Simple, early-retrieval-eligible	G2	8	20
Compound / multi-intent	G3	3	15
Single-question controls (must not be split)	Anti-over-fragmentation (Pitfall 5)	2	10
Late-arriving detail / refinement	G5	0	10
Presentation-only (reformat/shorten/summarize)	G2 false-trigger, G5 suppression	2	10
Out-of-corpus / insufficient evidence	G4 uncertainty correctness	0	8

Store this as one versioned YAML/JSON fixture file — it doubles as the direct input to your automated benchmark runner.

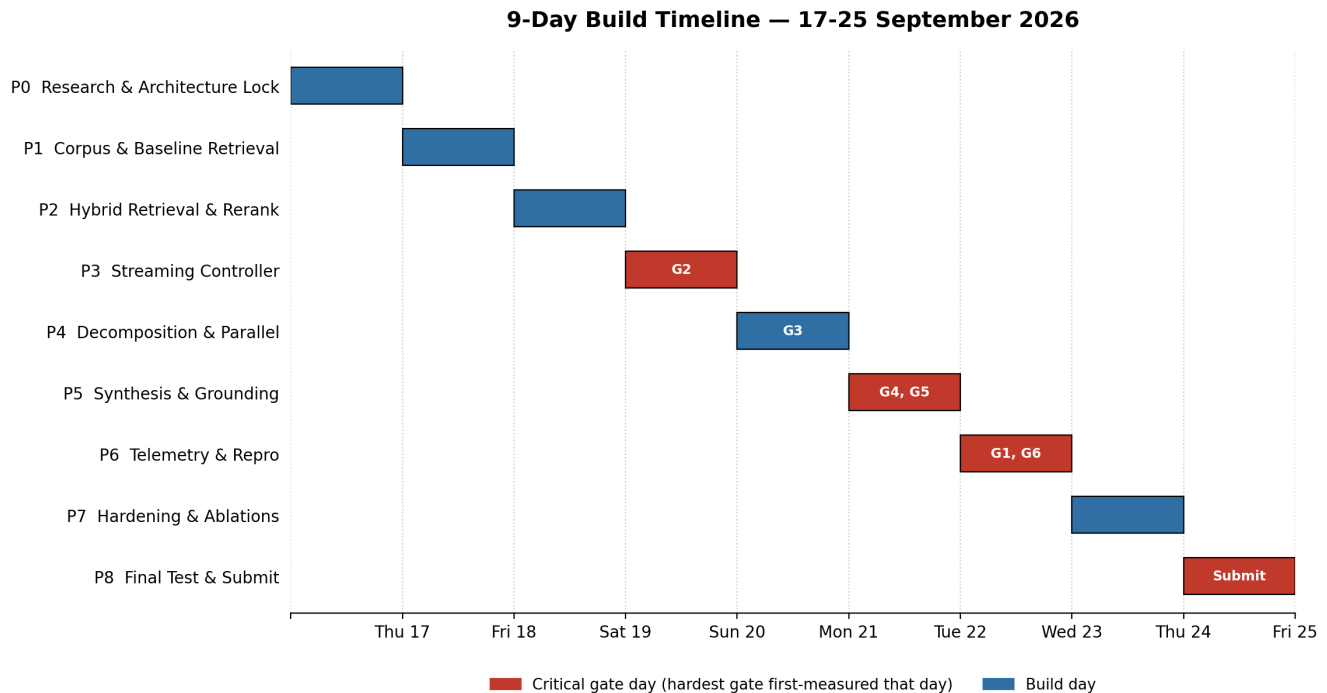
Metrics harness. One command (run_eval.sh / make benchmark) replays the entire labeled set through the live API and emits a single scorecard covering every G1-G6 number plus the supplementary engineering metrics in §12. This is the “automated replay suite” G1 asks for — build it once, run it dozens of times.

8. Gate Map: G1-G6 -> Phases

Gate	Threshold (guide's exact wording)	Primary build phase	First measured	Re-verified
G1 Reproducibility	Pass/Fail — single-command launch on a clean machine; automated replay completes unattended	Phase 0 (skeleton)	Phase 0 (informal)	Phases 6, 8 (formal)
G2 Early Retrieval	≥ 80% of eligible queries retrieve before transcript end; low false-trigger rate	Phase 3	Phase 3	Phases 6, 7
G3 Multi-Intent Identification	≥ 70% of compound queries; ≥ 2 distinct sub-intents isolated	Phase 4	Phase 4	Phases 6, 7

Gate	Threshold (guide's exact wording)	Primary build phase	First measured	Re-verified
G4 Factual Grounding	$\geq 85\%$ citation support; zero fabricated Doc_IDs	Phase 5	Phase 5	Phases 6, 7
G5 Session Refinement	Verified state continuity — narrows/updates without clearing session or re-running full-corpus search	Phase 5	Phase 5	Phases 6, 7
G6 Telemetry & Observability	100% trace coverage — timestamps, triggers, citations, version lineage, token cost	Phase 6	Phase 6	Phase 8

9. Day-by-Day Phase Plan



Every phase below follows the same shape: **Objective -> Exact Tasks -> Deliverables -> Tests/Validation -> Pass Gate**. Do not start the next phase's tasks until the current phase's gate is checked off — the gates exist precisely so a bad Day-4 decision doesn't surface as a mystery bug on Day 8.

Phase 0 — Day 1 · Thu 17 Sep — Technology Research, Architecture Lock & Foundation

Objective: Lock the tech stack with evidence, not vibes; stand up the skeleton repo, Docker, and shared schemas everything else builds on; remove the corpus-availability risk from the critical path.

Exact tasks:

- Full team: read both source documents together; walk through §3 (cheat sheet above) out loud so everyone can recite the 5 hard rules and 5 pitfalls without looking.

- Track D leads the stack decisions using §5/§6 above as the starting brief; team sanity-checks against current docs (Qdrant docs, FastEmbed GitHub, your chosen LLM provider's docs) rather than trusting memory.
- **POC A — Qdrant hybrid round-trip:** boot Qdrant via `docker-compose`, create a collection with one named dense vector (FastEmbed `bge-small-en-v1.5`) and one named sparse vector (Qdrant-native BM25), ingest ~20 toy chunks, run one Query API call combining `prefetch(dense) + prefetch(sparse)` with RRF fusion. Record latency.
- **POC B — LLM provider smoke test:** send 5 sample prompts (one decomposition-style, one synthesis-style, one controller-classification-style) to your default provider (Groq) and, if time allows, a second provider as a quality baseline. Log time-to-first-token, total latency, and whether the JSON output actually parses.
- Track A: draft the placeholder dev corpus (two small document sets per §7) and chunk it with `Doc_ID/§Section` metadata.
- Track D: scaffold the repo per §14, pin all dependency versions, write `docker-compose.yml` booting Qdrant + an API stub, write `.env.example`.
- Track D: define and commit the three shared schemas as Pydantic models — `StreamChunk`, `TelemetryEvent` (§4.3), `SessionState`.
- Send the registration/corpus-availability email if not already resolved (§1); log it as the first entry in `docs/RISKS.md`.
- Write the 5 ADRs from §5 into `docs/adr/` (copy-adapt from this document, don't start from scratch).

Deliverables:

- `docs/adr/ADR-1..5.md` committed
- `docker-compose.yml` that boots cleanly with `docker compose up`
- Pinned `requirements.txt` / `pyproject.toml`
- `StreamChunk`, `TelemetryEvent`, `SessionState` schema files
- Placeholder corpus, raw + chunked, under `data/dev_corpus/`
- `docs/POC_RESULTS.md` with real latency numbers from POC A and POC B
- `docs/RISKS.md` started, corpus risk logged with an owner

Tests/validation:

- `docker compose up` succeeds from a fresh clone on every team member's machine (your first informal G1 rehearsal)
- POC A returns non-empty fused results in a measured time (target: comfortably under 200ms on toy data)
- POC B: chosen LLM returns syntactically valid JSON on at least 4 of 5 sample prompts

PASS GATE -> Phase 1:

- ☐ Every ADR has a decision, not a "TBD"
 - ☐ `docker compose up` boots clean for the whole team
 - ☐ Both POCs produced a number, not just "it worked"
 - ☐ All three shared schemas are committed and everyone is coding against them
 - ☐ Corpus risk has an owner and a placeholder corpus exists
 - ☐ Registration status confirmed; final-submission link confirmed received or explicitly chased
-

Phase 1 — Day 2 · Fri 18 Sep — Foundation: Corpus Ingestion & Baseline (Dense-Only) Retrieval

Objective: A working, deliberately unglamorous, end-to-end path — text in, chunked and indexed, dense-only retrieval, un-grounded but functional LLM answer out. This is both your walking skeleton and the literal “baseline” the guide’s own deliverables checklist requires you to benchmark against later.

Exact tasks:

- Track A: finalize the chunking function (heading-aware where possible, fixed-size+overlap fallback); ingest the dev corpus into Qdrant with dense vectors only; write `/ingest` as an idempotent, folder-driven script (never a hardcoded file list — this is what makes the later corpus swap free).
- Track B/C: build the FastAPI skeleton with one synchronous `/turn` endpoint: full utterance in -> embed -> dense top-k search -> naive prompt-stuffing -> LLM call -> answer out. No citations, no controller, no streaming yet — deliberately the simplest thing that works end to end.
- Track D: wire the `TelemetryEvent` logger into `/turn` now, even though most fields stay empty/default this early — the habit is worth more than the completeness at this stage.
- Whole team: hand-write 15 example queries against the dev corpus with an expected `Doc_ID` noted for each — the first slice of your eval set (category: “simple, early-retrieval-eligible”).

Deliverables:

- `retrieval/ingest.py` + a populated dense-only Qdrant collection
- `/turn` endpoint answering via `curl`/Postman end to end
- First 15 labeled eval-set entries committed to `eval/labeled_set.yaml`
- First real `TelemetryEvent` JSON lines on disk

Tests/validation:

- Run all 15 hand-written queries through `/turn`; a human spot-checks that $\geq 10/15$ retrieve the expected `Doc_ID` (a sanity bar, not a graded gate — it just confirms the foundation isn't broken)
- Confirm `Doc_ID/$Section` metadata survives ingestion -> retrieval -> is visible in the raw response payload

PASS GATE -> Phase 2:

- ☐ `/turn` answers all 15 queries without crashing
 - ☐ $\geq 10/15$ retrieve a chunk with the correct `Doc_ID`
 - ☐ Every call to `/turn` produces one telemetry JSON line
 - ☐ `docker compose up` is still the only setup command required
 - If dense-only retrieval is clearly broken (wrong documents, not just imperfect ranking), fix chunking/embeddings now — don't layer hybrid complexity on a broken foundation.
-

Phase 2 — Day 3 · Sat 19 Sep — Hybrid Retrieval, Fusion & Reranking

Objective: Upgrade Phase 1's retrieval into the theme's actual required “Corpus Retrieval & Fusion” component — dense + sparse hybrid, RRF fusion, cross-encoder rerank, dedup — and get your first required ablation (hybrid vs. dense-only) as a natural byproduct.

Exact tasks:

- Track A: add a native Qdrant sparse (BM25) named vector alongside the existing dense one; re-ingest; calibrate `avg_len` to your actual chunk lengths rather than leaving defaults (ADR-2's flagged gotcha).
- Track A: implement the hybrid query — `prefetch(dense) + prefetch(sparse)` fused with RRF in one Query API call.
- Track A: add cross-encoder reranking (FastEmbed TextCrossEncoder, ms-marco-MiniLM-L-6-v2) over the fused candidates; dedup near-identical chunks retrieved by both arms before reranking.
- Track A: wire this into `/turn`, replacing Phase 1's dense-only call.
- Track D: build `eval/bench_retrieval.py` — runs the growing eval set through (a) dense-only and (b) hybrid+rerank, reports `recall@k` / hit-rate / latency for both. This is Ablation #1; save results now rather than reconstructing them under pressure on Day 8.

Deliverables:

- Hybrid Qdrant collection (dense + sparse) live
- `retrieval/hybrid_search.py` exposing: sub-query -> ranked, deduped, reranked chunk list
- `eval/bench_retrieval.py` + `docs/ablations/hybrid_vs_dense.md` (draft, real numbers)

Tests/validation:

- Retrieval benchmark runs clean across the whole current eval set with no errors
- Compare hybrid+rerank against dense-only: if hybrid does *not* win, investigate before assuming it should (check `avg_len` calibration first) — a documented negative result is still a valid, reportable finding
- P95 retrieval+rerank latency measured and logged

PASS GATE -> Phase 3:

- ☐ Hybrid query returns fused, deduped, reranked results for every eval-set entry without errors
- ☐ Ablation #1 has real, saved numbers
- ☐ P95 retrieval+rerank latency is known (target: keep this to roughly 60–70% of your eventual total per-turn budget, generation gets the rest)

Phase 3 — Day 4 · Sun 20 Sep — Streaming Controller & Early Retrieval (-> G2)

Objective: Build the theme's actual novel, graded mechanic: an incremental transcript-chunk simulator plus a controller that decides Wait / Retrieve / Suppress before the utterance ends, with every decision logged. Treat this as the highest-risk day in the whole plan.

Exact tasks:

- Track B: build `streaming/stream_simulator.py` — takes a full utterance and a chunking cadence (e.g., emit every 300–500ms or on clause boundaries) and re-emits it as (`t_offset_s`, `partial_text`) events, exactly matching the guide's own "Incremental transcript chunking simulator" roadmap item.
- Track B: implement the controller as a light decision function, not a heavy agent — start with rule-based heuristics (minimum token count, clause/punctuation boundaries, slot/entity completeness) to decide when an utterance looks "stable enough to check," then call your fast LLM (Groq) with a small structured prompt only at that point, classifying `{retrieve_now, wait, no_retrieval_needed}` with a short reason — mirroring the guide's own `retrieval_required: false, reason: "..."` output shape.
- Track B: log a `retrieval_event` (timestamp, trigger reason) on every controller firing.
- Whole team: extend the eval set with the "presentation-only" and "single simple question" categories from §7, each hand-labeled with the correct trigger behavior (or explicit no-retrieval).

- Track D: build `eval/bench_controller.py` — computes the G2 percentage (early-trigger rate on eligible cases) and the false-trigger rate on no-retrieval-labeled cases.

Deliverables:

- `streaming/stream_simulator.py`
- `controller/decide.py` with logged decisions
- Eval set extended to ≥ 25 –30 utterances across categories
- `eval/bench_controller.py` output: a real G2 percentage + false-trigger rate

Tests/validation:

- Run the full labeled set through simulator + controller; compute G2 live.

PASS GATE -> Phase 4:

- ☐ G2 $\geq 80\%$ on the current internal eval set — tune heuristic thresholds now if below; this gate gets harder to retrofit the longer you wait
 - ☐ False-trigger rate on no-retrieval-labeled cases is measured and reported as a number
 - ☐ Every controller decision carries a timestamp and a reason in the log
-

Phase 4 — Day 5 · Mon 21 Sep — Multi-Intent Decomposition & Parallel Retrieval (-> G3)

Objective: When the controller says "Retrieve," decide whether the utterance is one question or several, split it into orthogonal sub-queries without over-fragmenting, and retrieve + fuse across all of them concurrently.

Exact tasks:

- Track B: implement the decomposer as one structured LLM call (JSON list of sub-queries, single-element if not compound), with explicit few-shot guidance against splitting simple questions (Pitfall 5) and a hard cap (e.g., 3–4 sub-queries).
- Track B: wire decomposition output into Phase 2's hybrid retrieval using `asyncio.gather` so sub-query searches run concurrently, not sequentially — this is also your main time-to-first-token lever.
- Track A: merge/dedup the per-sub-query result lists into one evidence pool (two sub-queries can legitimately surface the same chunk).
- Whole team: add ≥ 15 compound-utterance cases (each hand-labeled with its true sub-intents) and confirm the single-question controls from Phase 3 are still in the set as an anti-over-fragmentation check.
- Track D: build `eval/bench_decomposition.py` — G3 percentage (≥ 2 correctly identified sub-intents on compound cases) plus an over-fragmentation rate on the single-question controls.

Deliverables:

- `decompose/decompose.py`
- End-to-end wiring: controller -> decomposer -> N parallel hybrid retrievals -> merged evidence pool
- Eval set extended with compound + single-question-control cases
- `eval/bench_decomposition.py` output: real G3 percentage + over-fragmentation rate

Tests/validation:

- Run the full set, compute G3
- Verify parallelism is real: total sub-query retrieval latency should track the *slowest* sub-query, not the sum of all of them

PASS GATE -> Phase 5:

- ☐ $G3 \geq 70\%$ on compound cases
 - ☐ Over-fragmentation rate on single-question controls is low and explicitly reported (set and justify your own bar — the guide flags this as a named pitfall but doesn't give a numeric threshold, so document what you chose and why)
 - ☐ Parallel retrieval is measurably concurrent, not accidentally sequential
-

Phase 5 — Day 6 · Tue 22 Sep — Session-Aware Synthesis, Grounding & Refinement (-> G4, G5)

Objective: Turn the evidence pool into a properly grounded, cited, versioned answer; add session memory so late-arriving detail refines rather than restarts; add presentation-only suppression. This is the densest, most failure-prone day — it's where Pitfalls 2, 3, and 4 all live at once — so protect the time for it.

Exact tasks:

- Track C: implement the synthesis prompt — evidence pool (with `Doc_ID/$Section`) + conversation history in, an answer with inline `[Doc_ID $Section]` citations and an explicit uncertainty field for any unsupported sub-intent out, mirroring the guide's own output example exactly.
- Track C: implement the deterministic grounding validator (ADR-5) — parse every cited tag, confirm it exists in that turn's actually-retrieved chunk set, flag/regenerate on any mismatch. This is your primary G4 mechanism.
- Track C: implement `session/store.py` — ephemeral, in-memory, keyed by `session_id`, holding turns / current answer text / current citations / `answer_version`. No persistence across restarts, no cross-session keys (satisfies the Session-Bound State rule by construction).
- Track C: implement the refinement classifier — on each new turn, classify as `NEW_TOPIC` / `LATE_DETAIL_ON_PREVIOUS` / `PRESENTATION_ONLY`:
 - `LATE_DETAIL` -> retrieve only for the new constraint, merge new citations with prior ones, regenerate only the affected claims, increment `answer_version`
 - `PRESENTATION_ONLY` -> zero retrieval calls, just reformat existing session content, log `retrieval_required:false` with reason
- Whole team: add ≥ 10 late-detail-refinement cases (modeled on the guide's own travel-reimbursement example) and ≥ 10 presentation-only cases to the eval set.
- Track D: build `eval/bench_grounding.py` (G4: citation-support %, fabricated-ID count) and `eval/bench_session.py` (G5: version increments correctly, no full re-retrieval on refinement, suppression fires with zero retrieval calls).

Deliverables:

- `synthesis/synthesize.py`, `synthesis/grounding_validator.py`
- `session/store.py`
- Eval set extended with refinement + suppression cases
- `eval/bench_grounding.py` and `eval/bench_session.py`, both producing real numbers

Tests/validation: run the extended set; compute G4 and G5 live.

PASS GATE -> Phase 6:

- ☐ $G4 \geq 85\%$ citation support, zero fabricated `Doc_IDs` on the eval set
- ☐ G5: 100% of late-detail cases increment version and retain prior citations without a full re-retrieval; 100% of presentation-only cases fire zero retrieval calls

- ☐ Uncertainty is explicitly emitted (never silently dropped) whenever a sub-intent lacks corpus support
-

Phase 6 — Day 7 · Wed 23 Sep — Observability, Reproducibility, Demo UI & Full Gate Run (-> G1, G6)

Objective: Make everything the system has quietly been doing all week visible, lock reproducibility, and run all six gates together for the first time as one integrated system rather than component-by-component.

Exact tasks:

- Track D: finalize the `TelemetryEvent` schema to 100% field coverage (every stage's timestamp, retrieval triggers, sub-queries, citations, answer version, per-call token cost, end-to-end latency, time-to-first-token) — this is G6, directly.
- Track D: build `eval/validate_schema.py` — checks a batch of logs for 100% required-field coverage.
- Track D (with input from B/C): build a minimal demo UI (plain HTML/JS over WebSocket, or Streamlit) that streams the simulated transcript, visually marks the moment the controller decides to retrieve, shows retrieved chunk IDs, streams the final answer with citations highlighted, and displays that turn's raw telemetry JSON in a side panel. This doubles as your Day-9 demo-video backbone.
- Track D: finalize `docker-compose.yml`, `.env.example`, and a real README with exact setup steps; write `eval/run_eval.sh` — one command that replays the entire labeled set through the *live* system and prints a full G1-G6 scorecard.
- Whole team: test the full stack from a genuinely clean checkout — fresh clone, fresh docker compose up, ideally on a machine that isn't a dev laptop (a teammate's machine, a clean VM, or a cloud sandbox).
- Run the first full G1-G6 scorecard as one integrated system and save it as `docs/BENCHMARK_REPORT.md v1`.

Deliverables:

- Complete telemetry schema + `validate_schema.py`
- Functional (not yet polished) demo UI
- `docker-compose.yml`, `.env.example`, README v1
- `eval/run_eval.sh` producing a full scorecard
- `docs/BENCHMARK_REPORT.md v1`

Tests/validation: clean-machine boot + full replay, end to end, unattended.

PASS GATE -> Phase 7:

- ☐ Clean-machine docker compose up + `run_eval.sh` completes with **zero manual steps** — this is G1, it's binary, and it's today's top priority above all else
 - ☐ G6 schema coverage is 100% on a sampled batch
 - ☐ A first full scorecard exists, even if some numbers are below target — Phase 7 exists specifically to fix what this run reveals, so an honest bad number today is far better than a surprise on Day 9
-

Phase 7 — Day 8 · Thu 24 Sep — Hardening, Ablations & Optimization

Objective: Convert Phase 6's scorecard into a punch list and clear it; complete both required architectural ablations properly; tune latency and cost. No new features start today.

Exact tasks:

- Whole team: triage the Phase 6 scorecard — rank every below-target gate by how far below target and how many labeled cases it affects; fix the worst offender first.
- Track A: finalize Ablation #1 (hybrid vs. dense-only retrieval) on the full, final eval set.
- Track B: run and write up Ablation #2 — **rule-based vs. model-based controller** (the guide's own suggested example): compare the Phase 3 heuristics-only controller against the heuristics+LLM-classifier version on G2 and false-trigger rate; report which wins and why.
- Whole team: identify and write up **at least 3 edge-case failures** (required by the guide's checklist) — e.g., a query with zero corpus support, a rapid double-correction in one session, a very long compound utterance — document what broke, what you changed, and what you deliberately chose not to fix and why.
- Track D: latency pass — profile the slowest turn types, parallelize anything still accidentally sequential, trim context sent to the LLM where possible; re-measure time-to-first-token and cost-per-turn before/after.
- End of day: freeze scope. Anything not done ships as-is or is explicitly cut — no exceptions, because tomorrow is submission day.

Deliverables:

- docs/BENCHMARK_REPORT.md v2, all gates re-measured
- docs/ablations/hybrid_vs_dense.md (final) and docs/ablations/rule_vs_model_controller.md
- docs/edge_cases.md — ≥ 3 documented failures + mitigations (or reasoned non-fixes)
- Latency/cost table, before vs. after today's tuning pass

Tests/validation: full `run_eval.sh` re-run after every fix; confirm nothing regressed while chasing one gate.

PASS GATE -> Phase 8:

- ☐ All six gates meet or exceed target on the current internal eval set — or, if any still falls short, it's a *known, documented, explicitly triaged* gap heading into Day 9, never a surprise
 - ☐ Both required ablations are written up with real numbers, not placeholders
 - ☐ ≥ 3 edge cases are documented
-

Phase 8 — Day 9 · Fri 25 Sep — Final Testing, Demo Recording & Submission

Objective: Freeze the system, prove it one more time on a genuinely clean environment, record the demo, assemble every required artifact, and submit well before 11:59 PM — with real buffer, not at the wire.

Exact tasks:

- Code freeze by early afternoon; bug fixes only from that point, no features.
- Final clean-machine test: fresh clone -> `docker compose up -> run_eval.sh` -> confirm the scorecard matches Day 8's numbers.
- Record the ≤ 5 -minute demo video, scripted to explicitly show every item the guide's own Deliverables Checklist names: early retrieval triggering, multi-intent decomposition, late-detail refinement, presentation-only suppression, citation traceability, and runtime telemetry. Use the Phase 6 demo UI as the visual spine of the recording.

- Finalize the System Architecture Brief (≤ 6 pages: design rationale, retrieval trigger logic, decomposition strategy, data provenance, trade-offs, failure-mode mitigations) — draw directly on §4–§5 of this plan and your ADRs; this section of work is mostly assembly, not fresh writing, if you kept the docs updated all week.
- Finalize the Benchmarking & Evaluation Report (quantitative comparison vs. baseline, both ablations, ≥ 3 edge cases — all already produced in Phase 7).
- Build the submission PPT/PDF with the **exact** required filename `CollegeName_TeamName_Submission_ppt` and required content: Theme ID + project title + team details; problem statement in your own words; solution + architecture diagram; tools and tech stack used; innovation highlights, results, and limitations.
- Tag the final commit exactly `PRISM_GENAI_HACKATHON_Y2026`; verify every file referenced anywhere (PPT, README, video) is present **in that tagged commit**, not just on a branch or in an untracked local folder.
- Submit through the official Google Form; confirm receipt with time to spare.

Deliverables: everything listed in §13 below.

PASS GATE (final — this is the finish line):

- ☐ Tagged commit `PRISM_GENAI_HACKATHON_Y2026` exists and contains every referenced file
- ☐ Demo video ≤ 5 minutes, uploaded, link verified to work in a logged-out/incognito browser
- ☐ PPT/PDF follows the exact naming convention
- ☐ Google Form submitted, confirmation received
- ☐ All of the above done with buffer before 11:59 PM — not at 11:59 PM

10. Risks & Fallbacks

Risk	Likelihood	Impact	Mitigation
Official corpus arrives late or differs structurally from assumptions	Medium	High	Placeholder corpus + corpus-agnostic ingestion (§7); re-run the full eval set within hours of arrival; Phase 7-8 carry deliberate buffer
LLM API rate limits/cost during heavy benchmark replay	Medium	Medium	Cheap/fast model for controller + decomposition, stronger model reserved for synthesis only; cache LLM calls during dev; exponential backoff
Controller misses the G2 80% threshold	Medium-High	High	Full dedicated day (Phase 3); numeric gate before any downstream work starts; thresholds kept in config, not hardcoded, so they're fast to retune
Decomposer over- or under-fragments	Medium	Medium	Explicit single-question control cases; hard cap on sub-query count; few-shot guardrails against Pitfall 5

Risk	Likelihood	Impact	Mitigation
Citation hallucination	Medium	High	Deterministic ID-membership validator, not "the LLM said it was fine" (ADR-5)
Session bugs — state bleeding across sessions, or not persisting within one	Low-Medium	High	Minimal, directly testable session store; explicit isolation unit tests
Docker fails on a judge's clean machine	Medium	High (G1 is binary)	Test on a genuinely clean environment starting Phase 6, not only dev laptops; pin every dependency version
Team time crunch (exams, other commitments across 9 days)	Medium	Medium	Strict daily gate discipline; MVP-first scope (§11); parallel workstreams from Day 1
Held-out benchmark structurally different from the guide's 3 worked examples	Low-Medium	Medium	Keep logic general and config-driven — never pattern-matched to the guide's own examples (the No-Hardcoding rule exists precisely for this)
Reranker adds latency without a quality gain on your actual corpus	Medium	Low-Medium	Ablate reranker on/off on your own eval set (Phase 2) before assuming it helps — research shows this isn't guaranteed
Judge's machine lacks your LLM API key	Low-Medium	High (blocks G1)	Document required env vars precisely; default to a provider with a fast, free signup path (Groq) so obtaining a key is a 2-minute step, not a blocker

11. MVP vs. Advanced (MoSCoW)

Must-have — the system is not a valid submission without these (target: done by end of Phase 6, Day 7):

- Corpus ingestion with Doc_ID/§Section metadata, corpus-agnostic
- Hybrid retrieval (dense + sparse) with RRF fusion and reranking
- Rule-based controller with logged early-retrieval events
- LLM-based multi-intent decomposition with a fragmentation guard
- Grounded synthesis with citations and explicit uncertainty flagging
- Ephemeral session memory with delta refinement and presentation-only suppression
- Full structured telemetry matching the guide's own schema shape
- `docker-compose single-command boot`
- Automated `run_eval.sh` producing a G1-G6 scorecard

Should-have — required by the guide's deliverables checklist, targeted for Phase 7, Day 8:

- Both architectural ablations, written up with real numbers
- ≥ 3 documented edge-case failures

- Minimal demo UI
- A completed latency/cost optimization pass

Could-have — only if Phase 7 finishes early:

- Upgrading the controller from rule-based to a genuinely tuned model-based version (beyond just the ablation comparison)
- Langfuse trace dashboard for demo polish
- Ragas/DeepEval secondary faithfulness metric in the report
- bge-reranker-v2-m3 upgrade if your own data shows it wins
- Multilingual corpus support

Won't-have — explicitly out of scope for this build:

- Real ASR/speech input (guide permits simulation)
- Cross-session user profiles or persistent memory (forbidden by the Session-Bound State rule)
- Any model fine-tuning
- Production auth, multi-tenancy, or billing
- A native mobile/voice-hardware wrapper

12. Metrics Reference

Metric	Target	Where measured
G1 Reproducibility	Pass/Fail, unattended	Phases 6, 8
G2 Early retrieval rate	$\geq 80\%$ of eligible cases	Phase 3+
G2 False-trigger rate (no-retrieval cases)	As low as achievable, reported explicitly	Phase 3+
G3 Multi-intent identification	$\geq 70\%$ of compound cases	Phase 4+
Over-fragmentation rate (single-question controls)	Team-set bar, reported explicitly	Phase 4+
G4 Citation support	$\geq 85\%$, zero fabricated IDs	Phase 5+
G5 Session continuity	100% of refinement/suppression cases behave correctly	Phase 5+
G6 Telemetry field coverage	100%	Phase 6+
Retrieval recall@k (hybrid vs. dense-only)	Hybrid $\geq$ dense-only, or a documented reason why not	Phase 2 (Ablation #1)
P95 retrieval+rerank latency	$\sim 60\text{--}70\%$ of total per-turn budget	Phase 2+
Time-to-first-token	As low as your LLM provider allows; tracked every phase, optimized Phase 8	Phase 3+
Cost per turn (token estimate)	Tracked from Phase 6 telemetry onward, minimized Phase 8	Phase 6+

13. Final Submission Checklist — 25 Sep

Copied and organized directly from the brief's own submission mechanics — treat every unchecked box as a blocker:

- ☐ Team registration completed (deadline was 16 Sep 11:59 PM — confirm, don't assume)

- ☐ Final-submission Google Form link confirmed received
 - ☐ Working prototype code in a public or shared GitHub repo
 - ☐ README with reproducible setup instructions (docker compose up or an equally clean CLI runner), Docker files, and any other requirements spelled out
 - ☐ Release tag on the final commit named **exactly** PRISM_GENAI_HACKATHON_Y2026 — the tagged commit is what gets judged
 - ☐ Everything referenced anywhere in your submission (PPT, demo video, documentation) is physically present **inside that tagged commit**
 - ☐ Demo video, ≤ 5 minutes, hosted on YouTube or Drive, link tested in a logged-out browser
 - ☐ Presentation file (PPT or PDF) named **exactly** CollegeName_TeamName_Submission_ppt
 - ☐ PPT/PDF content includes: Theme ID, project title, team details; problem statement in your own words; solution and architecture diagram; tools and tech stack used; innovation highlights, results, and limitations
 - ☐ Exactly one submission per team, via the official Google Form
 - ☐ Team/PPT naming consistently uses CollegeName_TeamName everywhere
 - ☐ Submitted before 25 Sep 2026, 11:59 PM — with real buffer, not at the deadline
 - ☐ Query channel noted in case anything goes wrong at the last minute: prism@samsung.com
-

14. Suggested Repo Structure

```

.
├─ docker-compose.yml
├─ .env.example
├─ README.md
├─ pyproject.toml          # or requirements.txt, pinned
├─ api/
│   └─ main.py             # FastAPI app, /turn endpoint (REST + WebSocket)
├─ controller/
│   ├── decide.py          # Wait / Retrieve / Suppress
│   └─ heuristics.py
├─ decompose/
│   └─ decompose.py
├─ retrieval/
│   ├── ingest.py
│   ├── hybrid_search.py   # Qdrant prefetch + RRF
│   └─ rerank.py
├─ synthesis/
│   ├── synthesize.py
│   └─ grounding_validator.py
├─ session/
│   └─ store.py
├─ telemetry/
│   ├── schema.py          # Pydantic models
│   └─ sink.py
├─ streaming/
│   └─ stream_simulator.py
├─ eval/
│   ├── labeled_set.yaml
│   └─ run_eval.sh

```

```
|   |─ bench_retrieval.py
|   |─ bench_controller.py
|   |─ bench_decomposition.py
|   |─ bench_grounding.py
|   |─ bench_session.py
|   └─ validate_schema.py
└─ data/
    └─ dev_corpus/
└─ demo_ui/
    └─ index.html
└─ docs/
    └─ adr/
    └─ ablations/
    └─ RISKS.md
    └─ POC_RESULTS.md
    └─ BENCHMARK_REPORT.md
    └─ edge_cases.md
    └─ ARCHITECTURE_BRIEF.md
```

15. Sources Consulted (17 Sep Research)

Technology choices above were checked against current documentation and 2026 comparative reporting rather than assumed from memory, per the plan's own Phase 0 discipline. Worth re-checking yourselves given how fast this space moves:

- Qdrant hybrid search, fusion (RRF/DBSF), and sparse-vector guidance: qdrant.tech/documentation/search/hybrid-queries, qdrant.tech/documentation/search/text-search
- FastEmbed (Qdrant's embedding/rerank library): github.com/qdrant/fastembed, qdrant.tech/documentation/fastembed
- RAG framework overhead benchmarking (LangChain/LangGraph/LlamaIndex/Haystack/DSPy): AIMultiple's 2026 RAG frameworks benchmark
- BM25 library performance: bm25s.github.io, github.com/xhluca/bm25s
- Reranker latency/quality trade-offs: [sentence-transformers cross-encoder documentation](#); [2026 reranker comparison write-ups](#)
- RAG evaluation tooling: deepeval.com (DeepEval vs. Ragas), Ragas project documentation
- Groq pricing, free-tier limits, and latency: console.groq.com/docs, current Groq pricing pages

This plan is a starting point, not scripture — where your own Day-1 benchmarking disagrees with a recommendation above, trust your numbers on your actual corpus over any general-purpose guidance, including this document's.